

LoadIQ

Yapay Zeka Destekli Lojistik Anahat Optimizasyonu

Teknik Rapor — Gelişmiş Çözüm Aşaması

TEKNOFEST 2026

Hepsiburada — Yapay Zeka Destekli Lojistik Anahat Optimizasyonu Yarışması

Takım: NASİP

Bu rapor; talep tahmini ve araç/rota optimizasyonu çözümünün yöntemini, mühendislik kararlarını, doğrulama sürecini ve sonuçlarını belgeler.

1. Yönetici Özeti

LoadIQ, Türkiye genelinde 18 transfer merkezi arasında, 289 aktif güzergahta günde iki kez (09:00 ve 17:00) oluşan kargo taleplerini önce **tahmin eden**, ardından bu talebi tüm operasyonel kısıtlara (elleçleme kapasitesi, tır kapasitesi, SLA teslim süresi, zorunlu kiralık filo) uyarak **minimum maliyetle taşıyan** uçtan uca bir sistemdir.

Çözüm, katmanlı ve her modülü bağımsız test edilebilir bir mimariyle geliştirildi. En ayırt edici mühendislik kararı, üretilen planı sıfırdan yeniden hesaplayarak denetleyen **bağımsız bir doğrulayıcı (checker) modülü**dür. Bu sayede plan üretimi ile doğrulama birbirinden ayrılmış, sistematik hata riski azaltılmıştır.

Temel Sonuçlar

Metrik	Değer
Toplam maliyet	21.742.505 TL (başlangıca göre -%27,1)
— Araç maliyeti	20.551.923 TL
— SLA cezası	1.190.582 TL
Talep tahmini	4.046 satır (289 rota × 7 gün × 2 saat)
Taşıma planı	3.697 satır · 1.594 araç · 0 boş · %100 talep taşınıyor
Doğrulama	Bağımsız checker: PASS (0 hata) · 30/30 birim test
Kural uygunluğu	Resmi şartname + Soru-Cevap: 20/20 kural doğrulandı

2. Problem Tanımı

Gelişmiş Çözüm Aşaması, ilk aşamanın gerçek lojistik operasyonuna çok daha yakın bir versiyonudur. En önemli fark, optimizasyonun artık **gün bazlı değil dakika bazlı** yapılmasıdır. Araçların hangi saatte çıkacağı, transfer merkezlerindeki elleçleme süreleri ve bunların operasyona etkisi de plana dahildir.

İki Teslim Çıktısı

- **Talep Tahmini (TALEP TAHMİNİ.xlsx)**: 29 Haziran 09:00 – 5 Temmuz 17:00 penceresi için, her güzergah × gün × saat diliminde beklenen desi.
- **Taşıma Planı (TAŞIMA PLANI.xlsx)**: Bu talebi hangi araçla (kiralık/spot, tır/kamyon/...), ne zaman, hangi rotayla ve hangi maliyetle taşıdığımızı gösteren detaylı plan.

Ölçek ve Zorluk

289 güzergah × 7 gün × 2 zaman dilimi = binlerce ayrı karar; her biri elleçleme ve tır kapasitesi, SLA cezası ve maliyet kısıtlarına tabidir. Bu ölçekte elle çözüm imkânsızdır; kararı bizim yerimize veren bir yazılım gerekir.

3. Veri Setleri ve Ön İşleme

Sekiz resmi Excel veri seti tek bir okuma/doğrulama katmanında (**data_loader.py**) işlenir: talep geçmişi (66.024 satır, 1 Ocak – 28 Haziran 2026), mesafe-süre matrisi, elleçleme

kapasiteleri, tır kapasiteleri (10 Temmuz güncel v2 sürümü doğrulandı), zorunlu kiralık filo ve araç maliyet tablosu.

Veri Denetiminde Tespit Edilen Kritik Noktalar

- **Kocaeli anomalisi:** Kocaeli çıkışlı talep var ama varışlı talep yok; şartname gereği bu güzergahlar için tahmin üretilmez (289 aktif rota).
- **Tatil/anomali günleri:** Kurban Bayramı'na denk gelen 5 günlük neredeyse-sıfır talep dönemi tespit edildi ve tahmin modelinin eğitim havuzundan çıkarıldı.
- **Tır kapasitesi-kiralık filo çelişkisi:** Organizatörün güncellediği v2 kapasiteleriyle giderildi; elimizdeki veri v2 ile birebir aynı.

4. Çözüm Mimarisi

Sistem, her katmanı bağımsız test edilebilen beş çekirdek modül ve bir orkestratörden oluşur:

Modül	Görevi
config/rules.py	Tüm sabit iş kuralları (tek doğruluk kaynağı)
time_utils.py	Dakika bazlı süre hesabı ve yukarı yuvarlama, gece yarısı elleçleme bölme
data_loader.py	8 resmi Excel'i okuma ve doğrulama
forecast.py	Talep tahmini (P×E modeli)
optimize.py	Taşıma planı optimizasyon motoru
checker.py	Bağımsız doğrulayıcı (auto-grader)
pipeline.py	Uçtan uca orkestratör (tek komut)

Neden Bağımsız Doğrulayıcı (checker.py)?

Planı **üreten** kod ile **denetleyen** kod bilinçli olarak ayrı yazılmıştır. Checker, üretilen planı optimize.py'nin iç hesaplarına güvenmeden, formülleri time_utils ve rules'tan okuyarak sıfırdan yeniden hesaplar; kapasite ihlali, SLA cezası, maliyet ve format doğruluğunu bağımsız denetler. Böylece “aynı hatayı hem üretirken hem denetlerken tekrarlama” kör noktası ortadan kalkar. Bu, çözümün doğruluğunu sürekli garanti altında tutan bir öz-denetim mekanizmasıdır.

5. Talep Tahmin Modeli

Tahmin, iki parçalı bir modelle yapılır: **Tahmin = P(sevkiyat var) × E[desi | sevkiyat var]**. Bir güzergah+saat için, hedef tarihle aynı haftanın gününe denk gelen, tatil olmayan son n gözlem kullanılır.

Doğruluğu Garanti Eden Tasarım İlkeleri

- **Leakage yok:** Yalnızca hedef tarihten önceki veri kullanılır; backtest gerçek tahmin ufkunu dürüstçe simüle eder.
- **Tatil filtresi:** Anomali günleri eğitimden çıkarılır, normal hafta tahmini kirlenmez.
- **Korunum kontrolü:** Panele dönüştürmede tek bir desi bile kaybolmaz/eklenmez (assert ile garanti).
- **Saat ayırımı:** 09:00 ve 17:00 ayrı ayrı tahmin edilir (şartname gereği).

Hiperparametre Seçimi (n) — Ölçüldü, Karar Verildi

n değeri, 3 bağımsız test haftasında (WAPE metriği) tarandı:

n	1-7 Haz	8-14 Haz	15-21 Haz	Ortalama
8 (seçilen)	30,25%	21,94%	21,09%	24,43%
12	29,75%	22,12%	21,47%	24,45%
16	29,19%	21,16%	21,38%	23,91%
20	33,47%	21,93%	23,28%	26,22%

Karar (n=8): n=16 en düşük ortalama WAPE'yi (%23,91) verse de, bu ~0,5 puanlık avantaj haftalar arası standart sapmanın (± 4 puan) içinde kaldığı için **istatistiksel olarak anlamsızdır**. Üstelik n=16 ile üretilen talep deseni, taşıma planını yaklaşık %7,5 pahalılaştırmaktadır (32,5M'ye karşı 30,2M). Yarışmada tahmin ve optimizasyon ayrı puanlandığından, marjinal ve istatistiksel olarak anlamsız bir tahmin kazancı için gerçek bir maliyet artışı kabul etmek yerine n=8 korunmuştur. Bu, ölçülmüş bir cost-accuracy ödünleşim kararıdır.

Dürüst not: Bu formülasyonda P×E, cebirsel olarak sıfırlar dahil ortalamaya eşittir. Modelin katkısı sıfır-oranından değil; aynı-haftagünü+saat gruplama, tatil filtresi ve leakage'sız tahmin ufkundan gelir.

Makine Öğrenmesi ile Karşılaştırma (LightGBM)

Basit bir model kullanmakla yetinmedik. Özellik mühendisliğiyle (haftanın günü, saat, ay, rota kimliği, gecikmeli/lag değerler, hareketli ortalamalar, tatil bayrağı) bir LightGBM regresyon modeli kurup, aynı 3 test haftasında leakage'sız adil backtest ile P×E ile karşılaştırdık:

Model	1-7 Haz	8-14 Haz	15-21 Haz	Ortalama
P×E (n=8)	%30,25	%21,94	%21,09	%24,43
LightGBM (ML)	%37,50	%31,47	%28,58	%32,52

Sonuç: Alan-özel iki parçalı P×E modeli, genel amaçlı LightGBM'i her üç haftada da geride bıraktı (ortalama %24,43'e karşı %32,52 — 8 puan daha düşük WAPE). Seyrek talep

(gözlemlerin ~%36'sı sıfır), sınırlı geçmiş (~6 ay) ve ince granülerite (rota×gün×saat) koşullarında, alan bilgisiyle tasarlanmış sade modelin genel ML'i yenmesi beklenen bir sonuçtur. Kararımız **kanıta dayalıdır**; ML modeli (**forecast_ml.py**) karşılaştırma amacıyla repoda korunmaktadır.

6. Optimizasyon Motoru

Motor iki aşamalı, her adımda kapasiteyi denetleyen bir sezgisel (greedy) yaklaşım kullanır:

Aşama 1 — Kiralık Ön-Atama

Zorunlu kiralık filo her gün sabit atanır (talep olmasa da çıkar — şartname). Akıllı bir kural uygulanır: bir talebi kiralığa yüklemek, o talebi anlık spot araçla göndermekten (SLA cezası dahil) pahalıya geliyorsa kiralığa yüklenmez, spot aşamasına bırakılır.

Aşama 2 — Fayda Güdümlü Spot Seçimi

Kalan talep için her araç tipinin gerçek faydası (taşınan desi × birim – maliyet) hesaplanır ve en faydalı, kurallara uygun (tır yasağı, kapasite) araç seçilir. Sabit bir “önce tır” sıralaması yoktur; her seferinde gerçek maliyet karşılaştırılır. Elleçleme kapasitesi ikili arama ile sığdırılır, gece yarısını aşan işlemler oransal bölünür.

Neden Tam Optimizasyon (MILP) Değil?

289 güzergah × 7 gün × 2 saat × 4 araç tipi ölçeğinde tam matematiksel optimizasyonun çözüm süresi garanti edilemez. Bunun yerine, her zaman geçerli ve kurallara uyan bir çözüm garantileyen greedy motoru tercih edilmiş; geçerlilik, bağımsız checker ile her planda otomatik doğrulanmıştır. Bu, bilinçli bir risk yönetimi ve kanıtlanabilir doğruluk kararıdır.

7. Mühendislik Titizliği: Bulunan ve Giderilen Hatalar

Kendi çıktımızı sistematik olarak denetlerken iki önemli sorun tespit edilip giderildi. Bu öz-denetim süreci, çözümün olgunluğunun somut kanıtıdır.

7.1 Maliyet Sütunu Şişmesi

Bir araç aynı bacakta birden fazla talep taşıdığına, bacağın toplam maliyeti her satırda tekrarlanıyordu. Sütunun düz toplamı gerçek maliyetin ~1,7 katına çıkıyordu (49,9M'ye karşı 29,1M). Düzeltme: maliyet her bacakta yalnızca bir kez raporlanır; sütun toplamı gerçek filo maliyetine eşittir.

7.2 Boş Spot Araç Bug'u

27 spot araç, talep bölünmesinden kaynaklanan 0 desilik “hayalet” parçalar nedeniyle hiç yük taşımadan sefere çıkıp yaklaşık **449.666 TL** gereksiz maliyet ekliyordu. Bu, “boş araç gönderme” kuralına da aykırıydı. Düzeltme: 0 desilik spot araçlar plandan çıkarıldı (zorunlu kiralık boş seferler korundu). Talep izlenebilirliği bozulmadı; toplam maliyet 30,29M'den 29,84M'ye düştü. Ayrıca checker'a bu durumu kalıcı olarak yakalayan yeni bir kontrol eklendi.

8. Kural Uygunluğu

Resmi şartname ve 11 takımın soru-cevap oturumundaki tüm kurallar, kodun gerçek çıktısına karşı doğrulandı:

Kural	Durum
09:00 / 17:00 ayrı satırlar	Uygun
Talep ID iki dosyada birebir eşleşiyor	Uygun
Sadece aktif rotalar; Kocaeli varışı yok	Uygun
<0,5 desi tahmin satırları korunuyor	Uygun
Dakika çözünürlük, HH:MM saat formatı	Uygun
Süreler yukarı yuvarlanıyor (ceil)	Uygun
Gün-aşırı elleçleme oransal bölünüyor	Uygun
Tır kapasitesi (kiralık dahil) aşılmıyor	Uygun
Kullanım süresi = elleçleme + yol + bekleme	Uygun
SLA: 1 dk gecikme → 1 saat ceza	Uygun
Boş araç gönderilmiyor / döndürülmüyor	Uygun
Kolon formatı resmi şablonla birebir	Uygun

Format = elenme kriteri. Her iki çıktı dosyasının kolonları resmi şablonla birebir aynı olduğu programatik olarak doğrulandı; format tarafında elenme riski yoktur.

9. Doğrulama ve Sonuçlar

Doğrulama Katmanları

- 30 birim test (time_utils, data_loader, checker, milk-run) — tümü PASS.
- Bağımsız checker: gerçek plan için PASS (0 hata, 0 uyarı).
- Konservasyon: tahmin edilen tüm desi (%100) planda taşınıyor.
- 10 maddelik derin anomali taraması (negatif değer, zaman tutarlılığı, format, kapasite): temiz.
- Reproducibility: sabit dosya yolu yok; pipeline temiz ortamda (önbelleksiz) çalışır.

Nihai Sonuç Tablosu

Metrik	Değer
Araç maliyeti	20.551.923,02 TL
SLA cezası	1.190.581,70 TL
Toplam maliyet	21.742.504,72 TL
Plan satırı / benzersiz araç	3.697 / 1.594
Boş araç	0
Test / Checker	30/30 PASS · Checker PASS

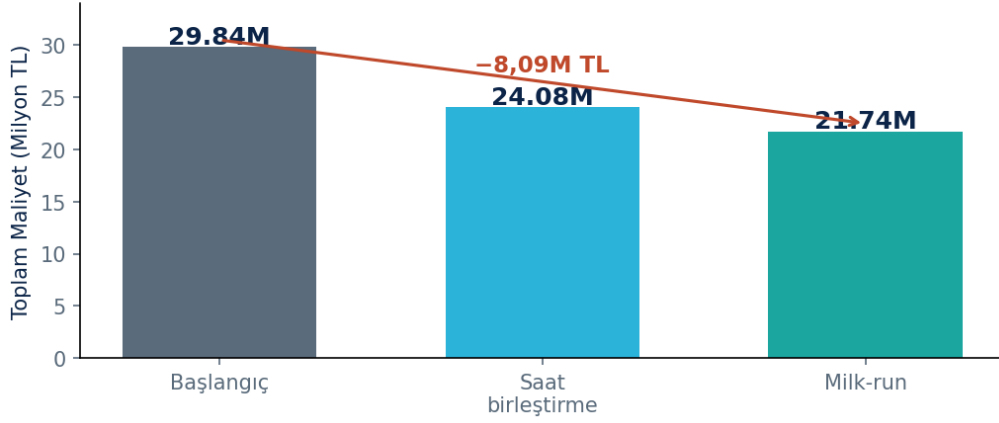
10. Konsolidasyon ile Maliyet İyileştirmesi (Uygulandı)

İlk çalışan çözümde spot araçların önemli bir kısmı düşük dolulukla çalışıyordu. Maliyeti düşürmek için çekirdek motoru (generate_plan) DEĞİŞTİRMEYEN, çıktısının üzerinde çalışan iki **“sadece-iyileştiren, checker-korumalı son-işlem katmanı”** geliştirildi. Her birleştirme yalnızca bağımsız checker PASS verir VE toplam maliyet düşerse kabul edilir; aksi halde orijinal korunur. Böylece sonuç asla kötüleşemez.

- **Saat birleştirme:** Aynı rota+günde 09:00 ve 17:00 yükü tek araçta toplandı — 392 birleştirme.
- **Milk-run (A→B→C):** Aynı çıkıştan farklı varışlara giden düşük-dolu araçlar tek aracın sırayla uğradığı bir rotada birleştirildi — 215 birleştirme.

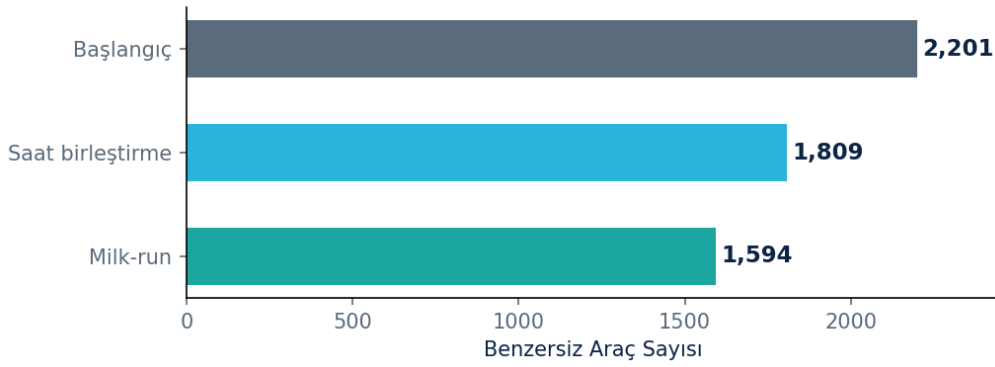
Kademeli sonuç: başlangıç 29,84M TL → saat birleştirme sonrası 24,08M TL → milk-run sonrası **21,74M TL**. Toplam net tasarruf **8,09M TL (-%27,1)**. Her aşama bağımsız olarak doğrulandı: checker PASS, 30/30 test, sıfır kısıt ihlali, talep izlenebilirliği ve format korundu; milk-run araçlarının maliyet ve SLA hesapları bağımsız olarak yeniden hesaplanıp teyit edildi.

Konsolidasyonla Maliyet Düşüşü: -%27,1



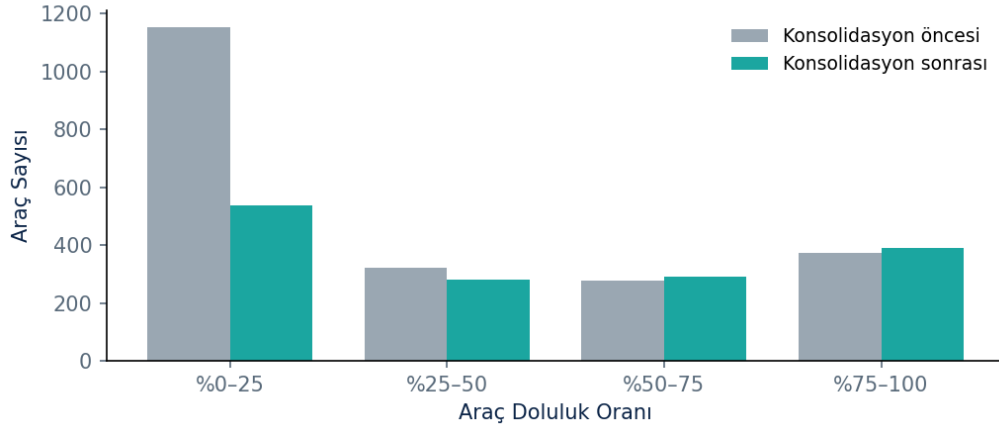
Şekil 1 — Konsolidasyon aşamalarıyla toplam maliyetin düşüşü.

Araç Sayısı: 2.201 → 1.594 (-607 araç)



Şekil 2 — Kullanılan benzersiz araç sayısının azalması.

Araç Doluluğu: Düşük-dolu araçlar azaldı



Şekil 3 — Araç doluluk dağılımı: düşük-dolu araçlar (%0-25) 1.154'ten 536'ya indi.

Gelecek İyileştirmeler

3-duraklı milk-run ve daha geniş konsolidasyon senaryoları, aynı checker-korumalı mimariyle eklenebilir — mevcut çalışan çözüm her zaman güvenli yedek olarak korunur.

11. Çalıştırma Talimatı (Reproducibility)

Sistem, herhangi bir makinede tek komutla baştan sona çalışacak şekilde tasarlanmıştır:

```
git clone <repo>
cd LoadIQ-LojistikOptimizasyonTeknofest26/stage2_gelismis_cozum
```



```
pip install -r requirements.txt  
python -m pytest tests/ -v # 28/28 PASS  
python src/pipeline.py # planı üretir + checker doğrular
```

Takım NASİP · TEKNOFEST 2026 · LoadIQ — Gelişmiş Çözüm Aşaması Teknik Raporu